

TP 13

Coloration d'un graphe – Sujet

Coloration d'un graphe

D'après CCINP-TSI 2024.

Introduction

La coloration d'un graphe consiste à attribuer une couleur à chaque sommet. Un k -coloriage donne une couleur (qui peut par exemple être un entier entre 0 et $k - 1$, pour simplifier) à chaque sommet, tel que deux sommets adjacents soient de couleurs différentes.

On peut appliquer ce problème au coloriage d'une carte de pays, ou d'un dessin d'ensemble (figure). Ainsi le coloriage de la pince (ainsi que le premier graphe) n'est pas un k -coloriage valide. Le second graphe en est un.

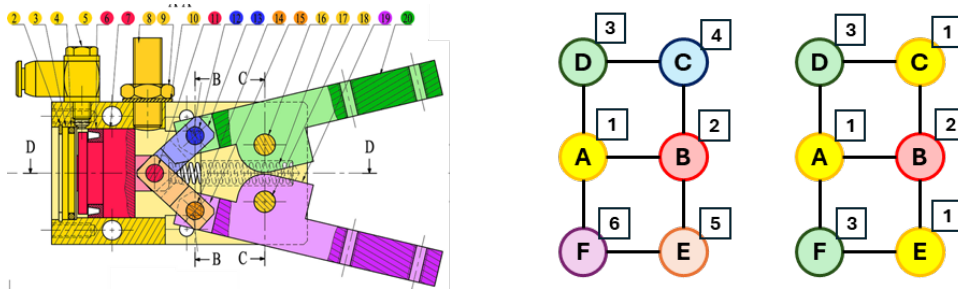


FIGURE 1 – Coloriage de la pince Schrader

Dans un premier temps, on modélise le graphe par un dictionnaire d'adjacence. Les clés sont les sommets, modélisés par des chaînes de caractères. Les valeurs sont des listes de chaînes de caractères.

Question 1 Donner le dictionnaire d'adjacence `Gex` associé au graphe de la figure 1.

Question 2 Implémenter la fonction `voisins(G:{}, s1:str, s2:str) -> bool` renvoyant `True` si `s1` et `s2` sont voisins, `False` sinon.

Une k -coloration est représentée par un dictionnaire `C` pour lequel les clés sont les sommets des graphes et les valeurs sont des entiers (entre 0 et $k - 1$) qui représentent la couleur du sommet.

On souhaite vérifier si une coloration est valide, c'est-à-dire que deux sommets du graphe reliés par une arête ne peuvent pas avoir la même couleur.

Question 3 Implémenter la fonction `coloration_valide(G:{}, C:{}) -> bool` renvoyant `True` si la coloration est valide, `False` sinon.

Algorithme intuitif de coloration

L'attribution des couleurs à chaque sommet est caractérisée par un dictionnaire `C` où `C[i]` est la couleur attribuée au sommet `i`. `C[i]` vaut -1 si la couleur n'est pas encore attribuée. Le dictionnaire `C` ne contient initialement que des -1 et ses valeurs sont modifiées progressivement, au fur et à mesure que les couleurs sont attribuées.

Question 4 Ecrire l'instruction permettant de créer un dictionnaire de n clés numérotées de 0 à $n - 1$ associées à la valeur -1.

Question 5 Implémenter la fonction `colore_sommet(G: {}, C: {}, s: str) -> None`. Cette fonction ne renvoie rien mais modifie le dictionnaire `C` en donnant à `C[s]` la plus petite couleur possible, en fonction des couleurs des sommets voisins qui sont déjà colorés.

Par exemple, pour le graphe `Gex`, prenons `C = [{"A": 0, "B": 1, "C": -1, "D": -1, "E": -1, "F": -1}]`. Les sommets "A" et "B" ont donc déjà été colorés avec les couleurs `C["A"] = 0` et `C["B"] = 1`. L'appel `colore_sommet(G, C, "C")` modifie le dictionnaire `C` en `C = [{"A": 0, "B": 1, "C": 0, "D": -1, "E": -1, "F": -1}]`. Cela veut dire que le sommet "C", adjacent au sommet "B" mais pas au sommet "A", a reçu la même couleur que le sommet "A".

Question 6 Ecrire une fonction `colorer1` avec pour argument un graphe (dictionnaire) `G` caractérisant un graphe, qui crée et renvoie le dictionnaire `C` des numéros des couleurs attribuées en colorant les sommets un par un par ordre croissant de leurs numéros.

Question 7 L'ordre de coloration imposé à la question précédente est arbitraire. On souhaite maintenant colorer le graphe en traitant les sommets selon un ordre arbitraire donné en argument. Écrire une fonction `colorer2` analogue à `colorer1` et avec un argument supplémentaire, une liste `ordre` fixant l'ordre de coloration des sommets.

Question 8 XXXX

La méthode que nous venons de décrire est rapide et fonctionne plutôt bien. Cependant, si on cherche à utiliser le nombre minimum de couleurs, l'efficacité de l'algorithme proposé ci-dessus dépend en grande partie de l'ordre dans lequel on choisit de colorer les sommets du graphe. L'objectif des sous parties suivantes est d'affiner la stratégie pour mieux choisir cet ordre de coloration.